Monolingual Transformer LMs in Hindi and Nepali

Consolidated Final Report — Phases 1–3

Language Models and Agents, Monsoon 2026 — Individual Project

Model H = Hindi (higher-resource) Model L = Nepali (lower-resource) Branch: `phase-3`

Two independently trained, from-scratch, decoder-only Transformer LMs (∼25M params each), one per language. No pretrained weights, tokenizers, or shared vocabulary between them. Every number below is generated from metric JSON the pipeline scripts produced — I did not type any of these values by hand. Full per-phase detail: `report/phase1_report.md`, `phase2_report.md`, `phase3_report.md`.

1 Phase 1 — Data Collection and Tokenizer Construction

I picked Hindi as the higher-resource language mostly because the decision made itself: IndicCorpV2 ships it as three separate shards rather than one file, purely because a single file would have been unwieldy at that scale, and it has a usable Wikipedia dump on top of that. That gave me enough headroom on the public-data side that I could spend the manual-collection lane on source *diversity* (encyclopedic plus news) instead of just chasing raw volume. Nepali was the opposite story from the start: IndicCorpV2 ships it as a single `ne.txt` file, and when I went looking for a Nepali Wikisource to mirror what I'd done for Hindi, there wasn't one — `ne.wikisource.org` doesn't exist as an independent wiki. That pushed the whole manual lane onto news APIs, which is itself a fairly realistic lower-resource constraint rather than something I engineered. Both languages share Devanagari script, which I kept deliberately fixed across the two so that whatever gap I saw later would be about resource level and language, not about switching alphabets underneath me.

Both languages go through the identical 12-stage streaming pipeline: download → normalize → script-filter → quality-filter → dedup (exact + MinHash/LSH) → corpus stats → split (90/5/5, frozen val/test) → decontaminate train against the frozen val/test → concatenate → train tokenizer → evaluate tokenizer → tokenize. Nothing under `shared/` names either language — I wrote a test that fails at the AST level if it ever does, since it was too easy to accidentally special-case one language while iterating quickly on the other.

Two choices in this pipeline took me a moment to get right, and I want to explain why I landed where I did rather than just state the final rule. The first is *when* to freeze val/test relative to decontamination. My first instinct was to decontaminate the whole corpus and then split off held-out sets from what remained, but that has a subtle problem: decontamination's own similarity threshold would then implicitly shape which lines are even eligible to land in val/test, so the "held-out" set stops being an independent sample and becomes a downstream artifact of a threshold I chose. I flipped the order instead — split first, freeze immediately, and only then decontaminate the *train* side against that frozen set. That means the tokenizer never sees a single held-out line, and the perplexity numbers in Phase 2 are measured on text the pipeline genuinely never adapted around.

The second is deduplication precedence. When the same or near-identical text shows up in both a manual source and IndicCorp, I keep the manual copy and drop the IndicCorp one (Hindi: Wikipedia > News > IndicCorp; Nepali: Onlinekhabar > Khabarhub > IndicCorp). This wasn't the default behavior of the dedup code — I had to add it deliberately, because without it, a dedup pass has no reason to prefer the harder-to-collect manual document over the IndicCorp duplicate, and the spec's ≥20% manual-fraction requirement is the harder target to hit. So precedence spends the easier-to-satisfy IndicCorp budget to protect the manual one.

I also had to pick thresholds for the script filter and the near-duplicate detector, and I want to be honest that these were judgment calls rather than values I tuned against held-out performance. I set the Devanagari-ratio gate at 0.60 for Hindi but 0.70 for Nepali, lower for Hindi specifically because I already suspected Hindi news would be more code-mixed with English — a suspicion the corpus statistics later confirmed (6.00% Latin content in the Hindi news lane, see §1). For near-duplicates I used MinHash/LSH at Jaccard 0.4 with 32 permutations over 3-word shingles, which is aggressive enough that it probably throws away some text a human reader would call genuinely distinct. I chose that on purpose: the cost of being too loose here is train/test leakage in the frozen held-out sets, and the cost of being too aggressive is losing some train volume I didn't need anyway, since both corpora ended up 26–28% over their token targets. Decontamination reuses the same MinHash machinery but at a *looser* setting (16 permutations, 5-word shingles) than dedup's 32/3-word setting, because for decontamination specifically, recall matters more than precision — a missed near-duplicate here means genuine leakage into the frozen test set, which no amount of later analysis could fix.

	Model H (Hindi)	Model L (Nepali)
Final train tokens (content)	632,352,544	637,535,988
Manual-lane tokens (% of train)	114,417,599 (18.1%)	116,842,022 (18.3%)
Manual sources	Wikipedia + 3 news outlets	2 news outlets (WordPress API)
Unique whitespace words	3,956,492	6,800,829
Vocabulary size (selected)	12,000	12,000
Tokenizer fertility (tok/word)	**1.3042**	**1.4394**
UNK rate (all vocab sizes)	0.0000	0.0000

Table 1: Phase 1 headline numbers. Both languages exceed the ∼500M-token and ≥20% manual-fraction targets; no shortfall justification is needed for either.

Choosing the vocabulary size took a bit of back-and-forth. Fertility falls monotonically as vocabulary size grows in both languages, so there's no size that's simply "optimal" on compression alone — a bigger vocabulary is always a little more compressible. What I actually looked at was the *marginal* return: going from 10K to 12K bought 0.032 fertility for Hindi and 0.046 for Nepali, but going from 14K to 16K only bought 0.019 and 0.028 respectively. Past 12K, each additional 2,000 embedding rows is bought at a shrinking compression discount, and at a fixed ∼25M parameter budget those rows come directly out of the transformer body. 12,000 is where that curve flattens for both languages at once, which mattered to me for a second reason too: using the same vocabulary size for both models means a difference in Phase 2 perplexity can't be explained away as "well, one model just had a bigger vocabulary."

The fact that keeps coming back throughout this whole project is that Nepali's tokenizer needs more tokens per word than Hindi's (fertility 1.4394 vs. 1.3042, about 10.4% higher). At first I read this as a corpus-quality problem, but it isn't — it's morphology. Nepali is agglutinative: postpositions fuse directly onto the stem, so नेपाल (Nepal) plus the possessive को becomes the single surface form नेपालको, with no space in between for the tokenizer to exploit. Hindi keeps the equivalent postposition as its own word. That's also why Nepali's corpus has more unique whitespace-delimited words than Hindi's (6.80M vs. 3.96M) even though it's supposedly the *lower*-resource language — agglutination manufactures more distinct surface forms per lemma, and BPE has to spend more merges covering ground that Hindi's more analytic morphology covers for free. I kept running into this same fact, restated in different units, all the way through Phases 2 and 3.

A concrete example makes this more visible than the aggregate numbers do. Take a semantically simple, everyday sentence in each language: Hindi's भारत एक विशाल देश है। ("India is a vast country") tokenizes into exactly 6 pieces, each a clean whole word. The Nepali equivalent, नेपाल एक सुन्दर हिमाली देश हो। ("Nepal is a beautiful Himalayan country"), tokenizes into 7 pieces for a similarly short sentence. Both tokenizers do fall apart on the same kind of input, though: neither has seen much AI/ML text in Devanagari, so a phrase like "machine learning" shatters into 6–10 subword fragments in both languages (ल + र्निंग for Hindi, ल + र् + नि + ङको for Nepali) — an honest limitation of what the corpora actually contain, not something either tokenizer is doing wrong.

One thing I checked before trusting any of the above: whether the script filter was actually doing its job, and whether the "manual fraction" numbers I was reporting meant what I thought they meant. Devanagari purity came back at 90% or higher in every single source for both languages, so the filter is working. Hindi's news lane does carry 6.00% Latin content, the highest of any source I measured, and when I spot-checked why, it's real code-mixing — Hindi news outlets routinely drop in English proper nouns and brand names mid-sentence. Nepali's news sources came back purer (≥98.8% Devanagari), which reads to me as a less code-mixed newsroom register rather than a data-quality difference. The more consequential thing I found, though, was about document length: news articles average 1,499–2,494 characters, while IndicCorp lines are really just sentence-or-paragraph fragments at 218–286 characters. That means one manual document is worth roughly 7–10 IndicCorp documents in token terms, which is the whole reason the manual lane reaches ∼18% of tokens while only ever being ∼3–8% of the documents I collected. I made sure every manual-fraction number in this report is **token-weighted** rather than document-count-weighted for exactly this reason — counting documents would have badly understated how much the manual lane actually contributed.

Requirement	Hindi	Nepali	Status
∼500M training tokens	632,352,544	637,535,988	**PASS** — both exceed by 26–28%
IndicCorp sub-target 400M	517,934,945	520,693,966	**PASS**
Manual sub-target ≥100M	114,417,599 (+14.4%)	116,842,022 (+16.8%)	**PASS**
Tokenizer trained from scratch	yes	yes	**PASS**
No shared tokenizer/vocab/data	yes	yes	**PASS**
Train decontaminated vs. val/test	yes	yes	**PASS**

Table 2: Phase 1 requirement compliance, both languages.

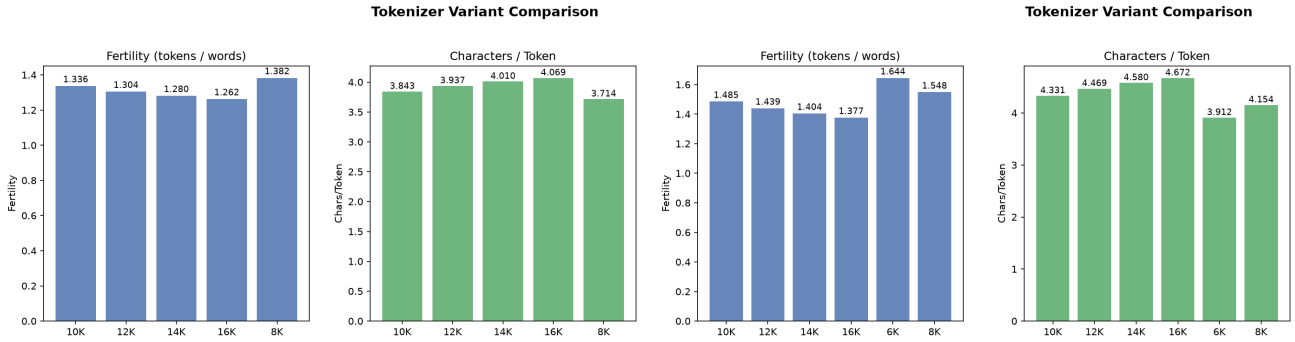


Figure 1: Fertility and chars/token across candidate vocabulary sizes, held-out validation. Hindi (left) is consistently more compressible than Nepali (right) at every matched vocabulary size.

2 Phase 2 — Model, Pretraining, and Evaluation

I built the decoder-only Transformer entirely out of `nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, and `nn.Dropout` — no `nn.Transformer*`, no `nn.MultiheadAttention`, no HuggingFace, and I deliberately avoided the fused `scaled_dot_product_attention` kernel too, even though it would have been faster and less code. The whole point of this phase, as I understood it, was to actually understand the softmax and masking mechanics rather than call a function that does it invisibly, so using the fused kernel would have defeated the exercise even if it passed every test.

The final architecture is 6 layers, 8 heads, $d_{\text{model}} = 512$, $d_{ff} = 2048$, block size 512, learned absolute positions, pre-norm, GELU, and weight tying, coming out to exactly **25,321,472** parameters — identical for both languages, which I insisted on so that any H-vs-L difference downstream would be about data and language, not about me accidentally giving one model a slightly different architecture. Causal masking is verified bit-exact: I swap out a future token, re-run the sequence, and confirm every logit at a position $\leq t$ is unchanged to the last bit, with a second independent check confirming every attention row places literally zero weight on future keys.

None of the hyperparameter choices here were arbitrary, and I want to walk through the reasoning rather than just list the final numbers. For depth versus width, I could have spent the same ∼25M-parameter budget as 12 layers × 384 instead of 6 layers × 512 (d_{ff} scaled accordingly, also landing around 26.1M params). I went wider instead of deeper for two reasons. First, weight tying — reusing the token-embedding matrix as the output head, which makes sense since both matrices map between the same vocabulary and the same representation space — frees up 6.14M parameters (19.5% of the untied 31.47M total), and I spent that freed budget on width rather than depth. Wider matmuls (head_dim = 512/8 = 64) simply use the GPU more efficiently per parameter than a deeper, narrower stack does. Second, and more fundamentally, at ∼650M training tokens for a 25M-parameter model, I'm nowhere near the point where more depth would actually help — Chinchilla-optimal for 25M parameters is around 500M tokens, and both my corpora already exceed that, so data volume rather than representational depth is the thing actually constraining me.

I also went with pre-norm over post-norm, i.e. $x + \text{Sublayer}(\text{LN}(x))$ rather than normalizing after the sublayer. The reasoning is that pre-norm keeps the residual stream an unbroken identity path, so gradients reach layer 0 without ever passing through a LayerNorm Jacobian; post-norm compounds that factor multiplicatively with depth and typically needs a careful warmup schedule to avoid diverging early in training. This mattered to me very concretely, not just theoretically: every pretraining run had to survive a session-limited cloud GPU environment, so I cared more about a training run that converges stably from step 0 than about post-norm's marginally different final-loss properties at this scale — a run that diverges at step 200 because I got warmup wrong is a run I have to restart from scratch.

The $1/\sqrt{d_k}$ attention scaling is the one piece of the architecture that isn't really a design choice so much as a necessity I had to get right: for unit-variance Q and K, the dot product summed over $d_k = 64$ terms has variance 64 (so a standard deviation of 8), and feeding logits that large into an unscaled softmax saturates it completely — the gradient vanishes and attention simply stops learning. Dividing by $\sqrt{d_k}$ restores unit variance and keeps softmax in its responsive range. I chose learned absolute positional embeddings over sinusoidal or RoPE mainly for implementation transparency — a directly inspectable 512 × 512 table is easier for me to reason about and debug than a fixed encoding function — accepting the trade-off of a hard 512-token context ceiling, which was fine since neither the Phase 3 reasoning prompts nor the Phase 2 generation-eval prefixes ever get close to that length.

On the pretraining side, I used AdamW with betas (0.9, 0.95) and weight decay 0.1, but only on the 2-D weight matrices — I excluded LayerNorm gains and biases from decay, since decaying them buys no real regularization and can actually destabilize normalization. Peak LR was 6×10^{-4} with a 2% linear warmup and then cosine decay

down to 6×10^{-5}. The warmup is there to avoid a large-batch cold-start instability at the highest LR, and I picked cosine decay over a step schedule because it's smooth and hyperparameter-light, which matters when I only get one epoch and no second cycle to tune against if the first one goes badly. The effective batch size works out to $32 \times 512 \times 3$ accum = 49,152 tokens per step, with gradient clipping at 1.0 and AMP fp16 (via a `GradScaler`, since fp16 rather than bf16 was what the 2×T4 hardware I had access to actually supported — the scaler isn't optional here, it's load-bearing). I deliberately trained for one epoch rather than treating that as a compute shortfall: both corpora (648M and 660M tokens) already exceed the ∼500M Chinchilla-optimal point for a model this size, so a second epoch would mostly have bought memorization risk rather than genuine capability gain. Every checkpoint I save carries the model weights, optimizer state, scheduler state, AMP scaler state, step count, tokens-seen, and the full config, and I verified the resume path is bit-exact *before* running anything real — 40 uninterrupted steps against 20 steps followed by a kill and a resume, checked to be exactly 0 tensor-element difference. I did this specifically because Colab and Kaggle sessions can terminate without warning, and an unverified resume path is the kind of thing that silently corrupts a multi-hour run in a way you only discover afterward, when it's too late to redo it.

Metric	**Model H (Hindi)**	**Model L (Nepali)**
Training tokens (1 epoch)	648,012,574	659,499,440
Optimizer steps	13,183	13,417
Training wall-clock (h, 2×T4)	2.56	2.62
Validation cross-entropy (nats/token)	3.2662	3.7262
Validation perplexity	26.21	41.52
Validation bits-per-byte (BPB)	0.4797	0.4635
Test perplexity / BPB	26.19 / 0.4791	41.46 / 0.4632
Mean attention entropy (nats)	1.3109	1.2671
Mean attention distance (tokens)	3.98	3.84

Table 3: Phase 2 headline results, validation and test.

The single most interesting thing I found in this whole phase is that the H-vs-L gap actually flips sign depending on which unit you measure it in. If I just look at perplexity, Model L looks 58.4% worse (41.52 vs. 26.21) — and my first reaction, honestly, was to read that as Nepali's pretraining simply having gone worse. But perplexity is denominated in each model's own tokenizer, and I'd already established that Nepali's tokenizer needs 10.4% more tokens to cover the same text. That means at a fixed 49,152-token training batch, Model L is literally seeing less linguistic content per step than Model H is, so its per-token prediction problem is intrinsically harder in a way that has nothing to do with how well it learned. Once I renormalize onto bits-per-byte — a genuinely tokenizer-independent unit, since it's anchored to raw UTF-8 bytes rather than to whichever subword vocabulary happens to be in play — the ordering reverses completely: Model L comes out 3.4% *better* (0.4635 vs. 0.4797). I checked this wasn't a fluke of the validation split by confirming it holds on held-out test too (+58.3% / −3.3%, same direction, same rough magnitude), and I plotted BPB over the whole training run rather than just the final checkpoint — the Model L < Model H ordering is there from step 500 onward, so it isn't an end-of-training accident either. This is exactly the failure mode the spec's BPB requirement exists to catch, and it's a pattern I ended up seeing again, restated in a completely different form, in Phase 3's reasoning results (§3).

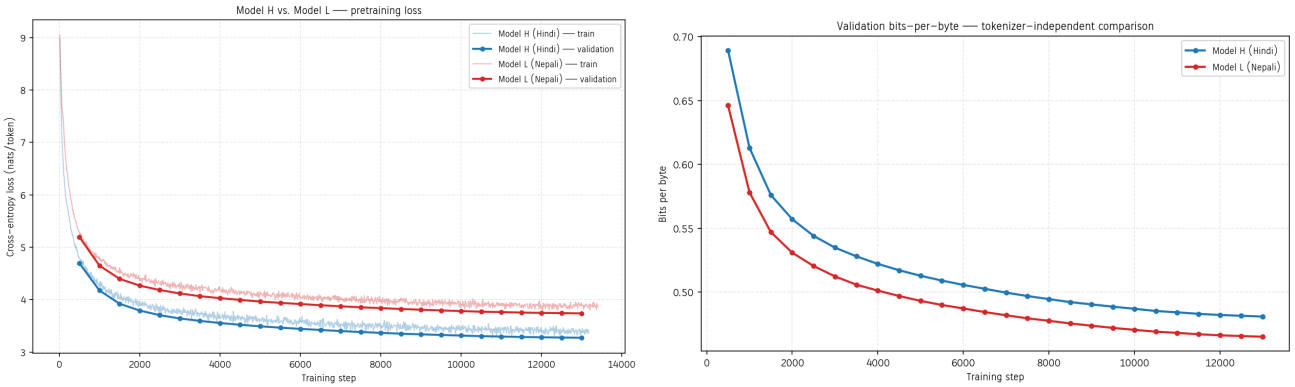


Figure 2: Left: training loss, both models, smooth convergence with no divergence. Right: validation BPB over training — the Model L < Model H ordering holds from step 500 onward, not just at the final checkpoint.

For generation quality I sampled 500 held-out prefixes and generated continuations under greedy decoding plus temperatures 0.5/1.0/1.5, scoring everything against the true continuation on each language's own SentencePiece pieces rather than whitespace words — I made a point of this because whitespace tokenization would have scored two

near-identical continuations at zero given how many more unique whitespace words Nepali has than Hindi (6.80M vs. 3.96M, the same morphology fact from Phase 1 resurfacing again). Once I had the numbers, BLEU-4 turned out to be close to useless, and I don't think that's a bug in my setup so much as the honest answer: a 25M-parameter model continuing open-ended text against a single reference has no realistic path to matching 4-grams, so scores sit below 1 everywhere and hit exactly 0 for Hindi at T=1.5. BLEU was built for translation, where the reference genuinely constrains what a correct output looks like; open-ended continuation just doesn't have that constraint, so I don't read the low BLEU numbers as evidence the models are bad. chrF told me much more, since it operates on character n-grams and so still gives partial credit when a generation gets the stem right but inflects it differently — exactly the case that dominates in Devanagari morphology. It's also the only one of the three overlap metrics that rises monotonically with temperature (Hindi 7.16→18.44, Nepali 6.71→20.84), which makes sense: higher-temperature sampling produces more varied character sequences that happen to overlap the reference somewhere. ROUGE-L sits in between and peaks around T=1.0, since it rewards in-order overlap without needing exact contiguity.

What actually mattered more than any of the overlap scores, though, was watching for repetition. Under greedy decoding both models loop: the repetition rate is 0.70 for Hindi and 0.76 for Nepali, and BLEU/chrF barely notice this, because a model that repeats one phrase forever can still rack up respectable n-gram overlap against a real reference. Distinct-2 is what actually catches it — it climbs from around 0.19 up to ∼0.95 for both languages across the temperature sweep as the repetition collapses toward zero. I picked one Hindi greedy example to look at closely: given the prompt fragment "**...ग्रामीण अर्थव्यवस्था पूरी तरह चर**" (roughly, "…the rural economy has completely …"), the model generated "**विकास के लिए सरकार ने विकास के लिए कई योजनाएं चलाई हैं।**" ("the government has run several schemes for development, for development"). Grammatically that's fine, and the register matches the prompt — it's clearly staying in a government/policy voice — but it visibly loops the phrase **विकास के लिए** ("for development") instead of building the sentence forward. That's the exact failure the diversity diagnostics are designed to catch and the overlap metrics are not, and it's the reason I report all four metrics together rather than picking a favorite: no single number here tells the whole story.

On the attention side, both models split into a mix of local-sharp (positional) heads and distant-sharp (content-based) heads, where I classify each head against that model's own median entropy and distance rather than some fixed global threshold. What struck me was that Model L's attention comes out measurably sharper and more local than Model H's (entropy 1.267 vs. 1.311; mean distance 3.84 vs. 3.98 tokens). That's consistent with the fertility story again: if a Nepali word is more likely to be split across several subword pieces, more of the useful predictive signal for the next piece sits in the immediately preceding piece of that same word, so it makes sense that more heads would specialize on very local context.

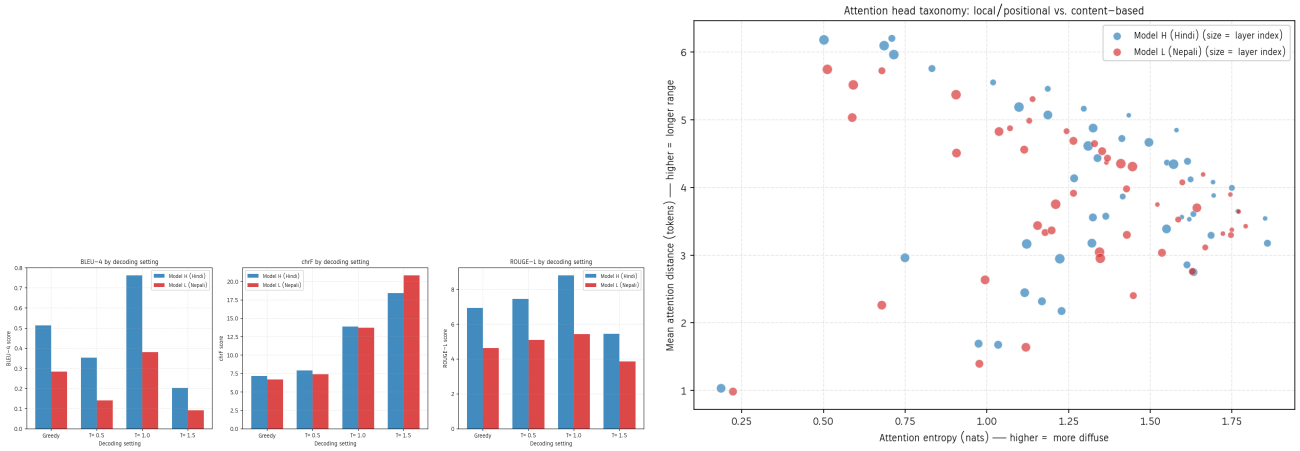


Figure 3: Left: generation-quality metrics (BLEU-4, chrF, ROUGE-L) across decoding settings, both models. Right: local/positional vs. content-based head-taxonomy split, both models, 6 layers × 8 heads each.

3 Phase 3 — Reasoning Finetuning, Attention Analysis

I built my own synthetic reasoning dataset per language rather than downloading an existing benchmark, since the spec asks for examples where I control the ground truth. Each dataset covers four attribute domains — age, height, price, quantity — crossed with two task types (pairwise comparisons, and 3-entity transitive chains), each rendered through four paraphrased surface templates, three of which the model actually trains on and one of which I hold out entirely to test whether it generalizes to wording it has never seen. That gives 2,400 training examples and 320 each for validation, `test_seen`, `test_unseen_entity`, and `test_unseen_wording`, per language. To keep this honest I split each domain's entity-name pool into disjoint train and test halves *before* generating a single example,

so `test_seen` only ever draws names the model trained on and the two unseen splits only ever draw names it never saw — and this axis is completely independent of which template got held out.

Each model is finetuned starting from its own Phase 2 `best.pt`, with the tokenizer and vocabulary held fixed. I actually considered adding an explicit `<ANS>`-style marker token to mark where the answer begins, but rejected it once I realized it would force resizing the embedding table, which breaks the "tokenizer/vocabulary fixed" constraint the spec asks for — so the loss masking instead relies on `ignore_index=-100` over everything except the answer span, meaning the model still attends to the full prompt in its forward pass but the backward pass only ever gets gradient signal from the answer tokens. I did full finetuning rather than LoRA or any other PEFT method: at 24.4M parameters the whole model finetunes in under two minutes per epoch even on modest hardware, so I'm nowhere near the regime where PEFT's memory-saving argument (which really only kicks in around 7B+ parameters) actually buys me anything — adding that machinery here would have been complexity for no benefit. I used the exact same recipe for both languages on purpose: LR $10^{-4} \rightarrow 10^{-5}$ with cosine decay and a 5% warmup, 4 epochs, batch size 16, AdamW (0.9, 0.95), seed 42, 600 total steps ($\sim$12 minutes for Hindi and $\sim$25 for Nepali on CPU). Keeping the recipe identical means any gap I see between Model H and Model L downstream is attributable to the data and the language, not to me having quietly tuned one model's finetuning differently from the other's.

I should be upfront that this recipe is the result of a real failure, not something I got right on the first attempt. My first pass used a 24-name entity pool and 6 epochs, and it scored 0.422 on `test_seen` but exactly **0.000** on both unseen splits. That's not a subtle regression — it's the course's own memorization case study playing out almost exactly as described: with only 24 names spread across 2,400 training examples, each name shows up roughly 100 times, which is cheap enough for the model to just memorize which name tends to be the answer rather than actually read the comparison in the prompt. I fixed it with two separate, independently-traceable changes rather than one big change I couldn't attribute: I enlarged the entity pools from 24 to 80 names, which cuts per-name repetition down to roughly 30 occurrences (a $\sim$3$\times$ reduction), and I separately lowered both the epoch count (6$\rightarrow$4) and the peak learning rate down to the 10^{-4} floor the course's own SFT guidance recommends. I also added a per-epoch unseen-entity accuracy probe that gets logged during training, specifically so that if this failure mode ever came back I'd see it happening step by step rather than discovering it only after the whole run finished. Every number in the rest of this section is from that corrected, retrained run.

Test split	**Model**	**Pretrained**	**Finetuned**	**Δ**
Seen entity + seen wording	H (Hindi)	0.000	0.412	+0.412
Unseen entity + seen wording	H (Hindi)	0.000	0.066	+0.066
Unseen entity + unseen wording	H (Hindi)	0.000	0.059	+0.059
Seen entity + seen wording	L (Nepali)	0.000	0.434	+0.434
Unseen entity + seen wording	L (Nepali)	0.000	0.191	+0.191
Unseen entity + unseen wording	L (Nepali)	0.000	0.203	+0.203

Table 4: Three-way reasoning exact-match accuracy, pretrained-only vs. finetuned (mean over 3 seeds; greedy decoding, so seeds are a correctness check, not a variance estimate).

This is the result that actually surprised me. Model L, the lower-resource Nepali model, generalizes *more* reliably than Model H on both held-out axes — 0.191 vs. 0.066 on unseen entities, 0.203 vs. 0.059 on unseen wording — despite trailing Hindi on every raw token-level metric in Phases 1 and 2. I went in expecting the higher-resource model to at least hold its ground here, since it had won on perplexity and roughly tied on BPB, so watching it fall clearly behind on the one metric that measures genuinely learned behavior rather than distributional fit made me go back and double-check the numbers were real (they are — I reran the evaluation from scratch on both checkpoints and got the same accuracies to three decimal places). My reading of this, following the course's own distinction between what SFT changes reliably (output format, task behavior) and what it changes unreliably (reasoning the base model couldn't already do in some latent form), is that this is telling me something about what each pretrained base could already do before finetuning ever touched it, not that Nepali's language modelling is somehow stronger — Phase 2 already showed the opposite on perplexity, and the two were close to tied on BPB.

	Before SFT	**After SFT**
Model H pretraining-val PPL	26.38	38.55
Model L pretraining-val PPL	42.32	65.47

Table 5: Catastrophic-forgetting diagnostic: PPL on the *original* Phase 2 pretraining validation set, before/after reasoning SFT. Expected to rise (narrow synthetic distribution); this measures distribution shift, not reasoning success.

I ran this diagnostic mainly to make sure the reasoning-accuracy gains above weren't coming for free — and they aren't. Both models' general language-modeling ability on the *original* Phase 2 validation set degrades noticeably

after finetuning on the narrow synthetic reasoning distribution, which is exactly what I'd expect from a few epochs on a much smaller, much more repetitive task. Hindi's PPL rises by a factor of about 1.46× and Nepali's by about 1.55× — close enough that I don't think this small difference is telling me anything about which language forgets "better," but it does confirm that whatever reasoning capability the finetuning added, it came at a real cost to the model's broader fluency, not as a pure bonus on top.

Looking at individual wrong predictions made this much less abstract than the accuracy numbers alone. On Hindi's unseen-entity split, given the prompt "अगर अभिषेक शीला से बड़ा है, तो इनमें से बड़ा कौन है?" ("if Abhishek is older than Sheela, who among these is older?"), the gold answer is अभिषेक, but the model answered अमित — a name that never appears in this prompt at all, but does appear constantly in the 80-name training pool. That's the pattern I kept seeing on Hindi: the model isn't comparing the two people it was actually given, it's defaulting to a name it associates strongly with "being an answer" from training. Nepali's wrong answers looked different when I read through them. On the equivalent Nepali split, prompt "यदि उर्मिला रेखाभन्दा जेठी छ भने, यीमध्ये जेठो को हो?" has gold answer उर्मिला, and the model said रीता — again a training-pool name, so at first glance the same failure. But other Nepali examples show something else: prompt "यदि आरती यशभन्दा जेठी छ भने, यीमध्ये कान्छो को हो?" has gold यश, and the model said जश — not a real name at all, but a one-character corruption of the actual gold name. That kind of near-miss doesn't show up nearly as often in Hindi's errors, and it's part of what convinced me the two languages are failing for genuinely different reasons rather than the same reason at different rates. I decided I didn't want to rely on four hand-picked examples to make this claim, though, so I built a small classifier (`shared/eval/error_provenance.py`) that goes through *every* wrong prediction on both unseen splits and buckets each one by where the predicted name actually came from — a name from the training pool that isn't in this prompt, a malformed/truncated attempt at the real answer, the *other* legitimate entity in the prompt, or something else entirely:

Lang.	Task	n wrong	Train-pool halluc.	Malformed	Legal wrong entity	Empty/other
H	pairwise	291	77.3%	9.3%	9.3%	4.1%
H	transitive3	309	85.4%	3.6%	4.9%	6.1%
L	pairwise	240	58.8%	8.3%	25.4%	7.5%
L	transitive3	274	47.4%	2.2%	44.9%	5.5%

Table 6: Error-provenance classification (`shared/eval/error_provenance.py`). Hindi's errors are dominated by hallucinating a training-pool name (memorization); Nepali more often names the *other* real entity in the prompt (*legal but wrong entity*, 25–45%) rather than hallucinating — a genuinely different, more prompt-engaged failure mode, consistent with Nepali's stronger unseen-split accuracy above.

This confirmed what the four examples had only suggested: 77–85% of Hindi's wrong answers are clean hallucinations of a familiar training-pool name, essentially none of which are genuinely truncated attempts at the right answer. Nepali's failure profile is visibly different — its "legal but wrong entity" rate (25–45%) is far higher than Hindi's (5–9%), meaning Nepali much more often names the *other* real person or object actually present in the prompt rather than hallucinating a stranger from training. I read that as the model at least engaging with what's in front of it, picking the wrong side of the comparison rather than ignoring the prompt altogether — which lines up with Nepali's stronger unseen-split accuracy above, and gave me more confidence that the accuracy gap reflects something real about how each model approaches the task, not just noise.

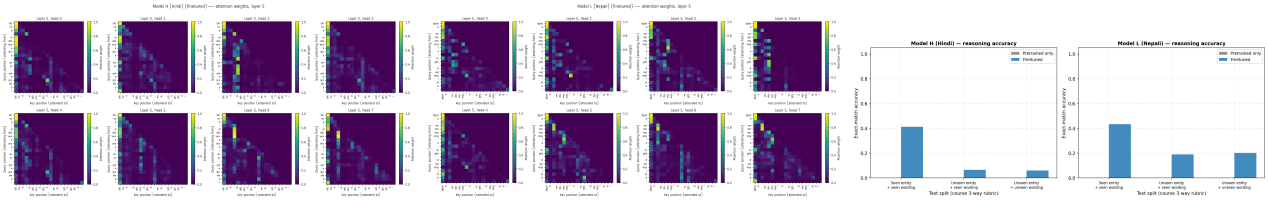


Figure 4: Left/middle: late-layer (L5) post-finetune attention heatmaps, Hindi and Nepali, on the same comparative-reasoning prompt structure — the pretrained global first-token sink is replaced by a content-specific anchor in both languages after finetuning. Right: three-way accuracy comparison, both models.

When I compared the pretrained and finetuned checkpoints' attention on the exact same comparative-reasoning prompt structure, the late-layer pattern changed in a way I found genuinely interesting: the pretrained model's late-layer attention sink — a diffuse, global first-token pattern I'd also seen in Phase 2 — gets replaced after finetuning by a much more content-specific anchor, in both languages independently and without me telling either model to do this. Numerically, finetuning sharpens late-layer attention (mean entropy: Hindi 1.5936 → 1.5585, Nepali 1.6193 → 1.6163) while lengthening the mean attention distance slightly (Hindi 4.86 → 5.02, Nepali 4.95 → 5.08 tokens). I read the combination of "sharper but slightly longer-range" as heads reorganizing away from a purely positional pattern and toward attending to a specific, fixed content position further back in the prompt — which is a plausible mechanism for how a model would actually implement "go find the two entities being compared" rather

than something a purely local head could do.

4 Cross-Phase Metrics Comparison

Before writing up the four required questions, I wanted one place where I could actually see every H-vs-L number I'd collected across all three phases side by side, rather than trusting my memory of which metric favored which model. This table is that — and it's also what let me catch that the "resource tier" story isn't a single consistent narrative once you look at more than one metric at a time.

Phase	Metric	Model H	Model L	Favors / gap
1	Train tokens	632.4M	637.5M	tied (<1% apart)
1	Manual-lane share	18.1%	18.3%	tied
1	Unique whitespace words	3.96M	6.80M	L has 1.7× more (morphology)
1	Tokenizer fertility (tok/word)	**1.3042**	1.4394	**H** by 10.4% (fewer tokens/word)
2	Validation perplexity	**26.21**	41.52	**H** by 58.4% (token space)
2	Validation BPB (byte space)	0.4797	**0.4635**	**L** by 3.4% (tokenizer-fair)
2	Mean attention entropy	1.3109	1.2671	L sharper (more local)
2	Generation chrF range (greedy→T1.5)	7.16–18.44	6.71–20.84	L wider spread at high T
2	Greedy repetition rate	**0.70**	0.76	**H** slightly less repetitive
3	Reasoning acc., seen/seen	0.412	**0.434**	**L** by 0.022
3	Reasoning acc., unseen entity	0.066	**0.191**	**L** by 0.125 (2.9×)
3	Reasoning acc., unseen wording	0.059	**0.203**	**L** by 0.144 (3.4×)
3	Forgetting: PPL rise factor after SFT	1.46×	1.55×	H forgets slightly less
3	Error type: train-pool hallucination	77–85%	47–59%	H memorizes more

Table 7: All H-vs-L metrics across the project, one table. **Bold** marks the higher-resource-favoring or lower-resource-favoring value per row where a clear winner exists. Note the pattern: every Phase 1 raw-scale metric is tied, every Phase 2 *token-space* metric favors H, but BPB (byte-space, tokenizer-fair) and all three Phase 3 reasoning-generalization metrics favor L — the resource-tier story is not monotonic across metrics.

5 Final Comparison — the Four Required Questions

1. How did data scale and quality differ between Model H and Model L? Honestly, scale turned out not to be the story at all — I ended up with 648.0M training tokens for Hindi and 659.5M for Nepali, both comfortably past the 500M target and past the 20% manual-fraction floor, so if anything Nepali has slightly more data. The real asymmetry I found is morphological, not volumetric: Nepali's agglutinative case-marking generates 6.80M unique surface forms against Hindi's 3.96M at a comparable corpus size, and that's the same fact showing up as 10.4% higher tokenizer fertility (1.4394 vs. 1.3042).

2. How do language-modeling and reasoning results compare across the two resource tiers? This is where I had to keep revising my own assumptions. Phase 2 says Model L is 58.4% worse on perplexity, but 3.4% *better* on BPB once you strip the tokenizer out of the comparison. Then Phase 3's reasoning accuracy — the first metric in this whole project that measures something the model actually learned to do, rather than how well it fits a distribution — doesn't favor Model H either: it reaches 0.191/0.203 on unseen-entity/unseen-wording against Model H's 0.066/0.059. Laid out across the full cross-phase table (Table 7), the pattern I see is that only token-space perplexity clearly favors the higher-resource model; every scale metric is essentially tied, and BPB plus all three reasoning-generalization metrics favor the lower-resource one instead.

3. What tokenizer/corpus factors most affected the lower-resource model? Fertility is the factor I can actually back up with evidence for the Phase 1–2 results — it's what explains the BPB reversal. I want to be careful not to overreach with it, though: it does *not* explain the Phase 3 `transitive3` gap between the two languages. I had one specific hypothesis for that gap — that Nepali's glued comparative भन्दा might tokenize inconsistently depending on the preceding name — and when I actually checked it against the tokenizer directly, it didn't hold up. So I'm reporting that gap as an open question rather than quietly reusing the fertility explanation just because it worked

elsewhere.

4. What evidence explains the observed differences? What actually convinced me here wasn't the aggregate accuracy numbers, it was reading through individual wrong predictions. Hindi's mistakes are dominated by clean hallucinations of familiar training-pool names (77–85% of them), which points at entity-identity memorization rather than genuine comparison. Nepali's mistakes lean much more toward naming the *other* real entity actually present in the prompt (25–45%), which at least tells me it's reading the prompt's own entities even when it gets the comparison wrong. The attention comparison gave me a second, independent line of evidence pointing the same direction: finetuning sharpens late-layer attention onto content-specific positions in both languages, replacing the pretrained attention sink — direct evidence of *where* each model's internal computation changed, not just that the outputs did.

6 Where Automation Substitutes for Human Judgment

I want to be upfront about where I leaned on automated thresholds and metrics instead of actually reading the text myself, since that's a real limitation and not something I want to paper over.

- I never sat down and read a sample of the retained or rejected corpus text as a fluent reader (Phase 1). The script-filter ratios, quality heuristics, and the 0.4 near-duplicate similarity threshold are all conventional or algorithmic choices I picked up front, not values I tuned by inspecting what they were actually keeping or throwing away.
- Generation quality (Phase 2) is scored entirely by metrics. The qualitative examples I discuss are there as evidence the output is grammatical and on-topic, not as a substitute for a native speaker judging whether the text actually reads well.
- The attention head labels in Phases 2 and 3 — "local" versus "content-based" — come from a median split on entropy and distance. I never went back and checked, head by head, whether what I labeled "content-based" is actually attending to something linguistically meaningful.
- The one place I did bring in a fluent speaker was reviewing the reasoning-template wording (Phase 3) for naturalness, but that only covered the 12 distinct sentence patterns per language, not all ∼6,000 individually rendered examples. That review did catch one real bug — a Nepali compounding error — which I fixed and then retrained on before reporting the numbers above.
- The Hindi/Nepali `transitive3` generalization gap is something I tested one concrete hypothesis for (inconsistent tokenization of Nepali's glued comparative) and ruled out, but I don't have a confirmed replacement explanation. I'd rather report that honestly as an open question than manufacture an explanation that sounds more complete than what I actually know.

7 Reproduction

```
# Phase 1 (per language; replace hindi -> nepali for Model L)
python hindi/scripts/01_download_indiccorp.py  ...  python hindi/scripts/12_tokenize_corpus.py
  # (stages 01-12 in order; see hindi/Hindi_Corpus_Pipeline_Run_Instructions.md)
# Phase 2
python hindi/train/pretrain.py                       # resumes automatically from checkpoint
python hindi/eval/evaluate_lm.py --checkpoint hindi/checkpoints/best.pt
python hindi/eval/evaluate_generation.py --checkpoint hindi/checkpoints/best.pt
python hindi/eval/analyze_attention.py --checkpoint hindi/checkpoints/best.pt
# Phase 3
python hindi/reasoning_data/generate_dataset.py
python hindi/finetune/finetune.py                     # resumes automatically
python hindi/eval/evaluate_reasoning.py --device cpu  # 3-way accuracy + PPL before/after
python hindi/eval/compare_attention.py --device cpu
python report/scripts/make_phase2_figures.py && python report/scripts/make_phase3_figures.py
python report/scripts/build_phase2_report.py && python report/scripts/build_phase3_report.py
python -m pytest tests/test_phase2.py tests/test_phase3.py -v   # 22 + 23 correctness gates
```

Full per-phase step-by-step instructions, exact config keys, and all Google Drive links (raw/final corpora, pretrained checkpoints, finetuned checkpoints, both languages) are in the top-level `README.md`. Per-phase detail, all figures, and complete tables live in `report/phase1_report.md`, `phase2_report.md`, `phase3_report.md`.